Detección temprana de incendios con YOLO11: detector, clasificador de recortes y criterio temporal sobre D-Fire

Martín Amagliani, Tatiana Durán, Tomás Frechou
Carrera de Especialización en Inteligencia Artificial (CEIA)
Facultad de Ingeniería, Universidad de Buenos Aires
Visión por Computadora II – Equipo 7

***Resumen*—Presentamos un prototipo de alerta temprana de incendios a partir de video que combina tres etapas: un detector YOLO11 de fuego y humo ajustado sobre D-Fire, un clasificador liviano que confirma o descarta cada recorte detectado y un criterio temporal K-de-N con estados normal, prealerta y alerta. Comparamos cuatro configuraciones del detector variando capacidad, resolución de entrada y presencia de imágenes negativas en el entrenamiento, y medimos latencia en CPU. El modelo *nano* a 640 px alcanza un mAP50 de 0,759 en test con 56 ms por frame en CPU; el modelo *medium* gana solo 0,020 de mAP50 a un costo cinco veces mayor. Entrenar con imágenes negativas reduce las imágenes sin incendio con algún falso positivo de 28,8 % a 0,65 % sin perder recall, y el clasificador de recortes las lleva a 0,10 % a cambio de 4,5 puntos de recall en humo. Además, mostramos que cerca de la mitad de las imágenes del test oficial de D-Fire tienen un casi-duplicado en entrenamiento, por lo que las métricas sobre ese split son optimistas.**

***Index Terms*—detección de incendios, detección de humo, YOLO11, D-Fire, falsos positivos, criterio temporal**

I. Introducción

Un incendio forestal detectado en sus primeros minutos puede contenerse con pocos recursos; detectado tarde, puede volverse incontrolable. Las redes de cámaras fijas en cerros y torres son una forma barata de vigilar grandes superficies, pero mirarlas en forma continua es inviable para operadores humanos. Por eso la detección automática de humo y fuego con visión por computadora se volvió un área activa [1].

El caso real más visible es el de California. Desde 2023, CAL FIRE y el programa ALERTCalifornia de la Universidad de California San Diego operan un sistema de IA desarrollado junto con DigitalPath que analiza más de 1.000 cámaras y avisa a los centros de despacho cuando detecta una anomalía compatible con humo. El piloto comenzó en seis centros de comando y en septiembre de 2023 se extendió a los 21 centros de despacho del estado [2]. En uno de los primeros casos, el sistema alertó un incendio iniciado a las 3 de la mañana en el Bosque Nacional Cleveland, que fue extinguido en 45 minutos [3]. Dos rasgos de ese sistema guían nuestro diseño: las alertas son verificadas por operadores, porque nubes, niebla y polvo generan falsos positivos, y lo que importa operativamente no es el mAP de cada imagen sino cuántas falsas alarmas por hora recibe un operador y cuánto tarda en llegar una alerta verdadera.

En Argentina, tras el incendio de El Bolsón de 2025, que afectó 221 viviendas, el Servicio de Prevención y Lucha contra Incendios Forestales (SPLIF) de Río Negro incorporó Alephfuego, una plataforma desarrollada por INVAP que reconoce columnas de humo en cámaras con cobertura de 360° y envía alertas al centro de monitoreo. El sistema opera en Bariloche y El Bolsón y se está extendiendo a otras localidades de la provincia [4], [5].

El objetivo de este trabajo es construir y evaluar un prototipo que, a partir de un flujo de video, emita alertas de fuego o humo con pocas falsas alarmas y latencia compatible con CPU. Nuestras contribuciones son:

- Un sistema de tres etapas (detector, clasificador de recortes y criterio temporal K-de-N) con código reproducible y una demo que procesa video en tiempo real.
- Un estudio controlado del detector que aísla el efecto de la capacidad del modelo, la resolución de entrada y las imágenes negativas, con métricas por clase y latencia en CPU y GPU.
- Evidencia cuantitativa de fuga entre los splits oficiales de D-Fire mediante *hashing* perceptual, y sus implicancias para interpretar las métricas.

II. Trabajos relacionados

D-Fire [6] es un conjunto de más de 21.000 imágenes anotadas con cajas de fuego y humo, con una fracción grande de imágenes negativas. Sus autores entrenaron detectores YOLOv4 y aplicaron poda de filtros para ejecutarlos en una Raspberry Pi 4, reduciendo el costo computacional hasta un 83,6 %. FIgLib y SmokeyNet [7] abordan el problema desde cámaras fijas de la red HPWREN: FIgLib reúne cerca de 25.000 imágenes etiquetadas de secuencias alrededor de cada ignición, y SmokeyNet explota la información espaciotemporal de varias imágenes consecutivas, con desempeño comparable al humano. Nuestro criterio temporal persigue el mismo objetivo con una regla determinística sobre las detecciones, sin un modelo temporal adicional.

Elegimos detectores de una etapa de la familia YOLO [8] en lugar de detectores de dos etapas como Faster R-CNN [9] porque el sistema debe procesar varios frames por segundo en CPU. Usamos YOLO11 [10], [11], preentrenado en COCO

[12], que ofrece variantes de distinta capacidad (*nano* a *x-large*) con la misma API para detección y clasificación.

III. Dataset y análisis exploratorio

Usamos la distribución de D-Fire en formato YOLO con su split oficial: 14.122 imágenes de entrenamiento, 3.099 de validación y 4.306 de prueba, con dos clases (*smoke* y *fire*). El 45,7 % de las imágenes son negativas, sin fuego ni humo (entre 44,4 % y 46,6 % según el split), y en total hay 14.685 cajas de fuego y 11.854 de humo.

Fuego chico, humo grande. Las dos clases tienen escalas muy distintas. La mediana del área relativa de una caja es 0,0054 para fuego y 0,142 para humo. Tras el *letterbox* a 640 px, el 45,0 % de las cajas de fuego es *small* según el criterio de COCO (área menor a 32^2 px), contra el 12,2 % de las de humo; la mediana del lado es 36 px para fuego y 186 px para humo. A 960 px la fracción de fuego *small* baja al 27,7 %. Esto motiva la hipótesis H1 sobre resolución.

Fuga entre splits. D-Fire incluye frames extraídos de videos y de cámaras fijas, por lo que dos imágenes pueden ser la misma escena sin ser idénticas byte a byte. No hay duplicados exactos (MD5) entre splits, pero al comparar un *difference hash* de 64 bits de cada imagen de prueba contra todo el entrenamiento, el 21,8 % tiene una imagen a distancia de Hamming 0 y el 49,9 % a distancia ≤ 4, que en la práctica es la misma escena. La fuga afecta tanto a negativas (61,1 % a distancia ≤ 4) como a imágenes con solo fuego (57,7 %) o solo humo (48,6 %). Mantenemos el split oficial para que los resultados sean comparables con la literatura, pero las métricas sobre él deben leerse como una cota optimista del desempeño en una cámara nueva.

IV. Metodología

La Fig. 1 resume el sistema. En inferencia, cada frame pasa por cuatro pasos.

Detector. YOLO11 ajustado sobre D-Fire, con cabeza de dos clases. Entrenamos 50 épocas con batch 16, semilla 42 y el aumento de datos por defecto de Ultralytics (mosaic, variación HSV, volteo horizontal y escalado), aplicado solo en entrenamiento. En inferencia usamos un umbral bajo ($conf_{\text{mín}} = 0{,}05$) para no descartar candidatos, NMS, y un umbral operativo τ que decide qué detecciones cuentan para la alerta.

Clasificador de recortes. Cada caja con confianza mayor que τ se recorta con un margen de contexto del 10 % por lado y se clasifica con un `yolo11n-cls` a 224 px en *real* o *confundible*. Su dataset se construye solo con imágenes de entrenamiento: los positivos son cajas anotadas y los negativos son falsos positivos del detector sin negativas (R5, ver abajo) sobre las 6.458 imágenes negativas de entrenamiento. Usamos R5 para minar porque, al no haber visto esas imágenes, sus errores se parecen a los que el sistema cometería en escenas nuevas. Las clases se balancean y la validación se separa por imagen. Una detección se *confirma* si además $p_{\text{real}} \geq 0{,}5$.

Criterio temporal K-de-N. Para cada clase se mantiene un buffer de los últimos N frames procesados. Si al menos K tienen una detección confirmada, la clase pasa a ALERTA y se emite un evento; con entre 1 y $K-1$ está en PREALERTA. Con histéresis, la clase permanece en alerta hasta que el buffer queda vacío, de modo que un incendio con detecciones intermitentes produce un solo evento. El estado global es el peor de las clases.

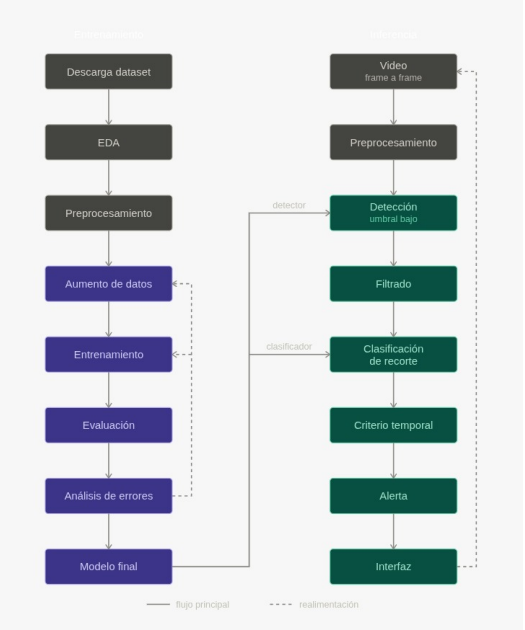


Figura 1. Pipeline del sistema. Izquierda: entrenamiento; derecha: inferencia sobre video. Las líneas punteadas indican realimentación.

Interfaz. La demo genera un video anotado con las cajas (punteadas si el clasificador las descarta), el estado, el contador K/N por clase, la latencia por frame y un registro de eventos (Fig. 5).

Diseño experimental. Entrenamos cuatro detectores para responder hipótesis concretas: R1 (*nano*, 640 px, con negativas) como base; R3 (*medium*, 640 px) para el compromiso capacidad–latencia; R4 (*nano*, 960 px) para H1, *aumentar la resolución mejora el recall de objetos chicos*; y R5 (*nano*, 640 px, sin las negativas de entrenamiento) para H2, *entrenar con negativas reduce los falsos positivos sin perder recall*. H3 plantea que *el criterio temporal reduce las falsas alarmas con una latencia de alerta acotada*, y H4 que *el clasificador de recortes reduce los falsos positivos con una pérdida de recall acotada*.

Además del mAP de Ultralytics, evaluamos cada sistema a un punto de operación fijo ($\tau = 0{,}4$): emparejamos predicciones y cajas anotadas de la misma clase con IoU $\geq 0{,}5$, de mayor a menor confianza, y reportamos precisión, recall y la fracción de las 2.005 imágenes negativas de prueba con al menos un falso positivo, que es la métrica más cercana a una falsa alarma. La latencia se mide en CPU (Intel Core i7 de 13.ª generación, batch 1, 100 imágenes de calentamiento y 300 medidas) y en GPU (NVIDIA L4).

Tabla I
DETECTORES EN EL SPLIT DE PRUEBA DE D-FIRE. LATENCIA POR IMAGEN: CPU CON BATCH 1 (BENCHMARK PROPIO); GPU L4, TIEMPO DE INFERENCIA POR IMAGEN REPORTADO POR ULTRALYTICS EN LA EVALUACIÓN.

		mAP50				ms/img	
Run	Config.	humo	fuego	mAP50	mAP50-95	CPU	GPU
R1	n, 640, neg	0,820	0,698	0,759	0,434	56	1,0
R3	m, 640, neg	0,838	0,719	**0,779**	**0,455**	299	5,9
R4	n, 960, neg	0,829	0,716	0,773	0,444	92	2,6
R5	n, 640, sin neg	0,727	0,670	0,699	0,402	55	1,0

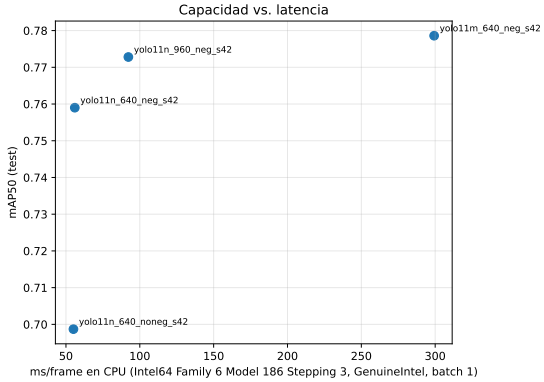


Figura 2. Compromiso entre mAP50 en prueba y latencia en CPU (batch 1) de los cuatro detectores.

V. RESULTADOS EXPERIMENTALES

V-A. *Capacidad, resolución y latencia*

La Tabla I resume los cuatro detectores. El fuego es la clase difícil en todas las configuraciones (mAP50 entre 0,06 y 0,12 por debajo del humo), consistente con su menor tamaño. Pasar de *nano* a *medium* mejora el mAP50 en 0,020 pero multiplica por 5,3 la latencia en CPU (de 56 a 299 ms, es decir, de 18 a 3 FPS). Como el criterio temporal necesita varios frames por segundo y una latencia mayor retrasa la alerta, elegimos *nano* como detector del sistema (Fig. 2).

H1 (resolución). R4 a 960 px supera a R1 en 0,014 de mAP50 con 1,65 veces su latencia. La Tabla II muestra que la mejora se concentra en los objetos chicos: el recall de fuego *small* sube 3,4 puntos y el de humo *small* 4,6, mientras que en objetos grandes no mejora (en fuego cae 2,6 puntos). H1 se confirma, pero el efecto es moderado: aun a 960 px, menos de la mitad del fuego chico se detecta, por lo que la resolución por sí sola no resuelve la detección de focos incipientes.

V-B. *Imágenes negativas (H2)*

Este es el efecto más fuerte del estudio. A $\tau = 0{,}4$, sin negativas en entrenamiento (R5) el 28,8 % de las imágenes negativas de prueba tiene al menos un falso positivo (0,33 por imagen); con negativas (R1) la cifra baja a 0,65 % (0,007 por imagen), una reducción de 44 veces. El recall prácticamente no cambia (humo 0,717 contra 0,737, fuego 0,554 contra 0,548) y la precisión en humo sube de 0,663 a 0,846. La Fig. 3 muestra que la curva de R1 domina a la de R5 en todo el rango de

Tabla II
H1: RECALL A $\tau = 0{,}4$ POR TAMAÑO DE CAJA (CATEGORÍAS COCO MEDIDAS A 640 PX) PARA R1 Y R4.

Clase	Tamaño	Cajas	R1 (640)	R4 (960)
fuego	small	1.318	0,405	**0,439**
	medium	1.137	0,653	0,656
	large	423	**0,747**	0,721
humo	small	302	0,487	**0,533**
	medium	490	0,618	0,637
	large	1.523	0,795	0,791

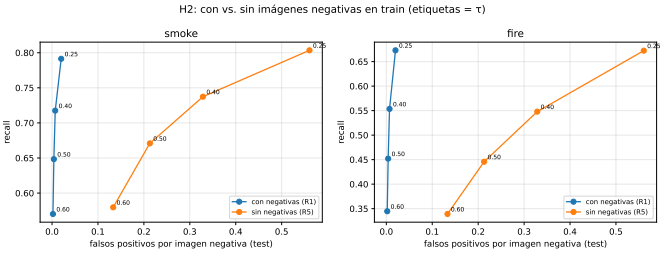


Figura 3. H2: recall contra falsos positivos por imagen negativa de prueba al variar τ, entrenando con (R1) y sin (R5) imágenes negativas.

τ: no hay un umbral al que R5 iguale a R1, de modo que la diferencia no es una cuestión de calibración. H2 se confirma. Hay que tener en cuenta que parte de esta ventaja proviene de la fuga descripta en la Sección III: R1 vio negativas casi idénticas a las de prueba.

V-C. *Clasificador de recortes (H4)*

En su validación (recortes de imágenes de entrenamiento separadas por imagen), el clasificador alcanza una exactitud de 0,942, con precisión 0,933 y recall 0,953 para la clase *confundible*. Sobre el test del detector R1 a $\tau = 0{,}4$, exigir $p_{\text{real}} \geq 0{,}5$ reduce las imágenes negativas con algún falso positivo de 0,65 % a 0,10 % (de 0,0070 a 0,0010 falsos positivos por imagen, siete veces menos), con una pérdida de 4,5 puntos de recall en humo (0,717 a 0,673) y 0,9 en fuego (0,553 a 0,544). Al variar el umbral del clasificador entre 0,1 y 0,9 se recorre ese compromiso (Fig. 4): con 0,8 no queda ningún falso positivo en negativas, pero el recall de humo cae a 0,634. El costo computacional es bajo: el sistema completo tarda 63 ms por frame en CPU (0,83 recortes por imagen en promedio), un 12 % más que el detector solo. H4 se confirma en el test oficial, pero como R1 ya casi no comete errores en esas negativas, el beneficio real del clasificador debe medirse sobre escenas externas a D-Fire (Sección VI).

V-D. *Sistema completo sobre video (H3)*

Procesamos dos clips de cobertura periodística de incendios forestales (60,5 s y 41,5 s, 50 fps) con el sistema completo (R1, clasificador, $\tau = 0{,}4$, $K/N = 3/5$). En ambos el sistema entra en alerta de humo en torno al primer segundo y de fuego antes de los 7 s, y el detector más clasificador corre a 30–40 FPS en la CPU de una notebook.

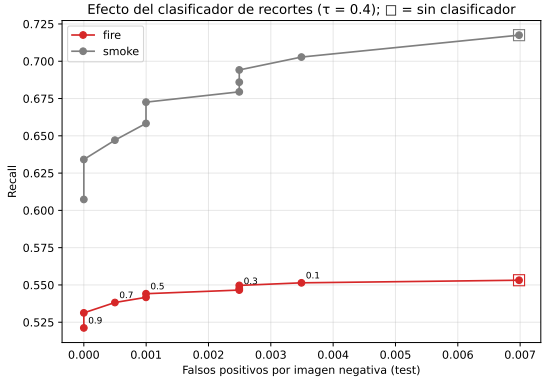


Figura 4. H4: recall por clase contra falsos positivos por imagen negativa de prueba al variar el umbral del clasificador (R1, $\tau = 0{,}4$). El cuadrado marca el sistema sin clasificador.



Figura 5. Demo del sistema sobre video: cajas de fuego (rojo) y humo (celeste) confirmadas por el clasificador, estado ALERTA, contador K/N por clase, latencia por frame y registro de eventos.

El experimento dejó una observación de diseño: N debe fijarse en tiempo y no en frames. Procesando todos los frames a 50 fps, la ventana de 5 frames cubre 0,1 s; el parpadeo natural de las detecciones vacía el buffer y el mismo incendio dispara una alerta nueva cada pocos segundos (49 eventos en los dos clips). Procesando uno de cada diez frames (5 fps, ventana de 1 s) los eventos bajan a 12, y en el segundo clip, una única escena continua, a uno por clase. El primer clip es un montaje de varias escenas, por lo que parte de sus eventos corresponde a cortes de edición.

Estos clips no permiten medir la latencia de alerta ni las falsas alarmas por hora, porque el incendio ya es visible desde el primer frame y no incluyen tramos sin evento. Por eso H3 queda evaluada solo de forma cualitativa.

V-E. Análisis de errores

Revisamos los 20 falsos positivos de mayor confianza y los 20 falsos negativos de mayor área por clase de R1 a $\tau = 0{,}4$. Ninguno de esos falsos positivos cae en una imagen negativa, y 5 son confusiones entre clases (humo detectado como fuego o al revés). En humo, una parte importante de los falsos positivos de alta confianza son predicciones correctas penalizadas por la anotación: el modelo marca toda la columna de humo mientras que el *ground truth* la divide en cajas más chicas o cubre solo una parte, de modo que el IoU queda por debajo de 0,5 (Fig. 6). Esto sugiere que las métricas de humo subestiman la utilidad del detector para la tarea de alerta, donde lo que importa es la presencia y no el contorno exacto.

VI. Conclusiones

1) **Las imágenes negativas son la decisión de datos más importante.** Incluirlas en el entrenamiento reduce 44 veces la fracción de imágenes sin incendio con falsos positivos (28,8 % a 0,65 %) sin pérdida de recall.
2) **Para CPU, el modelo *nano* es el punto de equilibrio.** El modelo *medium* mejora el mAP50 en solo 0,020 con 5,3 veces más latencia; *nano* permite procesar 18 FPS en CPU.
3) **La resolución ayuda poco con el fuego incipiente.** Pasar a 960 px mejora 3,4 puntos el recall de fuego chico a 1,65 veces el costo, y el recall de esa categoría sigue por debajo de 0,45: es el principal cuello de botella del sistema.
4) **El clasificador de recortes es un filtro barato y ajustable.** Con un 12 % de latencia adicional reduce siete veces los falsos positivos en negativas, a cambio de 4,5 puntos de recall en humo; su umbral es un parámetro operativo.
5) **El criterio temporal debe parametrizarse en segundos.** Una ventana definida en frames depende de la tasa de cuadros: a 50 fps no filtra nada, mientras que a 5 fps los eventos de alerta se reducen cuatro veces.
6) **El test oficial de D-Fire es optimista.** Cerca de la mitad de sus imágenes tienen un casi-duplicado en entrenamiento; recomendamos separar por cámara o escena al evaluar generalización.
7) **El mAP de humo subestima la utilidad para alerta.** Muchos falsos positivos de alta confianza son discrepancias de granularidad con la anotación, no errores de presencia.

Limitaciones y trabajo futuro. Todos los detectores se entrenaron con una sola semilla, por lo que diferencias menores a 0,01 de mAP no son concluyentes. La evaluación en video es cualitativa y usa clips periodísticos, no cámaras fijas de vigilancia. Queda pendiente medir falsas alarmas por hora y latencia de alerta sobre secuencias de cámaras fijas con el inicio del evento anotado, como las de FIgLib [7], y evaluar el clasificador sobre un conjunto de estrés externo con nubes, niebla, atardeceres, luces nocturnas y reflejos.

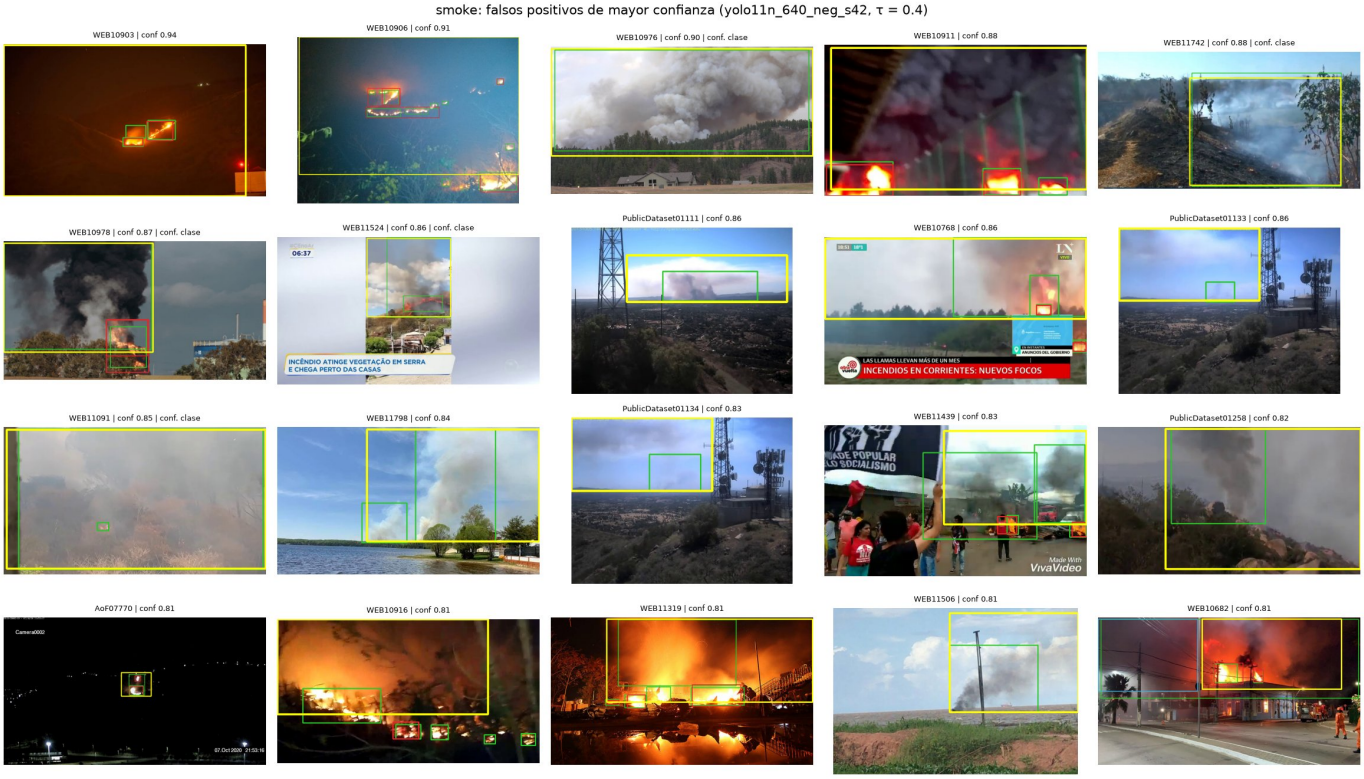


Figura 6. Los 20 falsos positivos de humo de mayor confianza de R1 ($\tau = 0{,}4$). En amarillo, la predicción contada como falso positivo; en verde, el *ground truth*; en rojo y celeste, otras predicciones de fuego y humo. En muchos casos hay humo visible dentro de la caja y la discrepancia es de granularidad de la anotación.

Referencias

[1] C. Jin, T. Wang, N. Alhusaini, S. Zhao, H. Liu, K. Xu, and J. Zhang, "Video fire detection methods based on deep learning: Datasets, methods, and future directions," *Fire*, vol. 6, no. 8, p. 315, 2023.

[2] ALERTCalifornia, "ALERTCalifornia and CAL FIRE's fire detection AI program named one of TIME's best inventions of 2023," https://alertcalifornia.org/alertcalifornia-and-cal-fires-fire-detection-ai-program-named-one-of-times-best-inventions-of-2023/, 2023, consultado: oct. 2026.

[3] Reuters, "California turns to AI to help spot wildfires," https://www.reuters.com/world/us/california-turns-ai-help-spot-wildfires-2023-08-11/, Aug. 2023, consultado: oct. 2026.

[4] Gobierno de Río Negro, "Cámaras con IA y tecnología satelital para la prevención de incendios," https://rionegro.gov.ar/articulo/57747/camaras-con-ia-y-tecnologia-satelital-para-la-prevencion-de-incendios, consultado: oct. 2026.

[5] Diario Neuquino, "Río Negro incorpora inteligencia artificial para detectar incendios y prepara la expansión del sistema a Conesa," https://diarioneuquino.com.ar/rio-negro-incorpora-inteligencia-artificial-para-detectar-incendios-y-prepara-la-expansion-del-sistema-a-conesa/, consultado: oct. 2026.

[6] P. V. A. B. de Venâncio, A. C. Lisboa, and A. V. Barbosa, "An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices," *Neural Computing and Applications*, vol. 34, no. 18, pp. 15 349–15 368, 2022.

[7] A. Dewangan, Y. Pande, H.-W. Braun, F. Vernon, I. Perez, I. Altintas, G. W. Cottrell, and M. H. Nguyen, "FIgLib & SmokeyNet: Dataset and deep learning model for real-time wildland fire smoke detection," *Remote Sensing*, vol. 14, no. 4, p. 1007, 2022.

[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[10] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," 2024.

[11] G. Jocher and J. Qiu, "Ultralytics YOLO11," https://github.com/ultralytics/ultralytics, 2024, versión 8.4.157 utilizada en este trabajo.

[12] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, ser. LNCS, vol. 8693, 2014, pp. 740–755.